DilithiAuth: A Post-Quantum Identity Provider Mitigating Token Bloat via Hybrid Signatures and Sidecar Reference Stores

Uttam Mahata *Department of Computer Science and Engineering*
DilithiAuth Research Project
Email: uttam@dilithiauth.org

July 28, 2026

Abstract

The imminent arrival of Cryptographically Relevant Quantum Computers (CRQCs) presents a critical threat to modern Identity Providers (IdPs) relying on classical public-key infrastructure. Adversaries currently execute "Harvest Now, Decrypt Later" (HNDL) attacks, capturing encrypted sessions and signed OAuth 2.0 / OpenID Connect (OIDC) tokens for eventual cryptanalysis using Shor's algorithm. While NIST has standardized post-quantum signature schemes such as ML-DSA (FIPS 204), transitioning IdPs to post-quantum signatures introduces severe practical challenges, primarily "token bloat." A single ML-DSA-44 signature adds 2,420 bytes of raw data, inflating JSON Web Tokens (JWTs) to nearly 4.7 KB and exceeding standard HTTP header limits (typically 4 KB). This paper introduces **DilithiAuth**, a research-oriented post-quantum IdP designed to mitigate HNDL threats while resolving token bloat. DilithiAuth combines classical Ed25519 with post-quantum ML-DSA-44 in a dual-signature hybrid architecture under a unified Key Identifier (`kid`). To accommodate diverse operational constraints, DilithiAuth introduces three distinct token issuance strategies: Thin JWT, Jumbo JWT, and CBOR Web Tokens (CWT). Through extensive empirical benchmarking, we demonstrate that Thin JWTs offload the 2.4 KB ML-DSA signature to a sidecar store (`SigStore`) via a 43-character SHA-256 reference (`pqc_ref`), reducing HTTP header overhead to 441 bytes while supporting fast-path classical verification in 43.37 μs. Furthermore, binary-encoded CWTs achieve a minimal token payload of 211 bytes (304-byte header overhead) with verification latency of 58.20 μs and only 1,729 bytes per operation. For zero-trust offline verification, Jumbo JWTs embed the full ML-DSA signature by increasing server header tolerance to 16 KiB. We detail the system architecture, mathematical size models, empirical benchmarks, and security guarantees against quantum forgery and sidecar tampering.

Post-Quantum Cryptography, Identity Provider, ML-DSA-44, FIPS 204, OAuth 2.0, OpenID Connect, Token Bloat, Hybrid Signatures, CBOR Web

Token.

1 Introduction

Modern web and cloud architectures depend heavily on federated identity protocols, specifically OAuth 2.0 [**?**] and OpenID Connect (OIDC) [**?**]. Identity Providers (IdPs) issue cryptographically signed assertion tokens—such as JSON Web Tokens (JWTs) [**?**]—that encode user identity, authorization scopes, and session validity. Verifiers, reverse proxies, and microservices validate these signatures to grant access across distributed microservices.

Today, nearly all deployed IdPs rely on classical public-key primitives such as RSA or Elliptic Curve Digital Signature Algorithm (ECDSA) / Edwards-curve Digital Signature Algorithm (Ed25519) [**?**]. However, Shor's algorithm [**?**] proves that a Cryptographically Relevant Quantum Computer (CRQC) can solve the discrete logarithm problem and integer factorization in polynomial time. Consequently, an adversary equipped with a CRQC can forge classical signatures, impersonate any user, and compromise federated identity domains.

This vulnerability is exacerbated by the "Harvest Now, Decrypt Later" (HNDL) attack paradigm [**?**]. Adversaries currently eavesdrop on network traffic and record encrypted session handshakes, Authorization headers, and signed identity claims. Even if quantum computers are years away from deployment, historical authentication tokens and long-lived authorization assertions captured today will be vulnerable to retro-active forgery and decryption once CRQCs emerge.

To counteract HNDL attacks, the National Institute of Standards and Technology (NIST) finalized its first suite of Post-Quantum Cryptography (PQC) standards in 2024, including FIPS 204 (ML-DSA) [**?**], based on algebraic lattice hardness. However, integrating post-quantum digital signatures into existing web transport infrastructure introduces a fundamental engineering obstacle: **token bloat** [**?**, **?**].

While classical Ed25519 produces compact 64-byte signatures and 32-byte public keys, ML-DSA-44 (NIST Security Category 1) requires a 2,420-byte signature and a 1,312-byte public key. When base64url-encoded within a standard JSON Web Token, a post-quantum identity token inflates from ≈ 400 bytes to over 4.6 KB. Standard web servers, API gateways, and reverse proxies (e.g., Nginx, Envoy, and default Go `net/http` servers) default to HTTP request header limits of 4 KiB to 8 KiB. Embedding full post-quantum signatures directly into HTTP `Authorization: Bearer` headers causes pervasive HTTP 431 (*Request Header Fields Too Large*) errors and severe network amplification.

To address these challenges, this paper presents **DilithiAuth**, an open-source, production-ready IdP designed for quantum-safe identity management. DilithiAuth makes the following primary contributions:

1. **Dual-Signature Hybrid Architecture**: We implement a hybrid signing engine (`pkg/signer`) combining classical Ed25519 with post-quantum

ML-DSA-44 under a unified Key Identifier (`kid`), maintaining backwards compatibility while providing lattice-based quantum resistance.

2. **Tri-Strategy Token Pipeline**: We introduce three customizable token issuance models (`pkg/oauth`): *Thin JWT*, *Jumbo JWT*, and *CBOR Web Token (CWT)* [**?**].

3. **Sidecar Reference Store**: For Thin JWTs and CWTs, we decouple the 2,420-byte post-quantum signature from the HTTP transport header by storing it in a sidecar signature repository (`pkg/transport/SigStore`) referenced by a 43-character SHA-256 digest (`pqc_ref`).

4. **Empirical Performance Benchmarks**: We provide a comprehensive performance evaluation detailing exact byte sizes, HTTP header overhead, signing and verification latency, and memory allocations across all primitives and token strategies.

2 Background & Related Work

2.1 NIST PQC Standards & FIPS 204 (ML-DSA-44)

In August 2024, NIST formally published FIPS 204 [**?**], standardizing the Module-Lattice-Based Digital Signature Algorithm (ML-DSA), derived from the CRYSTALS-Dilithium scheme [**?**]. ML-DSA security is grounded in the hardness of the Module Learning With Errors (M-LWE) and Module Short Integer Solution (M-SIS) problems over polynomial rings.

ML-DSA defines three parameter sets corresponding to NIST security categories: ML-DSA-44 (Category 1, equivalent to AES-128), ML-DSA-65 (Category 3, equivalent to AES-192), and ML-DSA-87 (Category 5, equivalent to AES-256). DilithiAuth adopts ML-DSA-44 as its target post-quantum primitive, offering an optimal balance between security and byte efficiency.

TABLE **??** compares the cryptographic parameters of Ed25519, ML-DSA-44, and the resulting hybrid scheme.

Table 1: Cryptographic Parameters of Primitives in DilithiAuth

Primitive / Scheme	**Public Key**	**Private Key**	**Signature**
Ed25519 (RFC 8032)	32 B	64 B	64 B
ML-DSA-44 (FIPS 204)	1,312 B	2,560 B	2,420 B
Hybrid (Ed25519 + ML-DSA)	1,344 B	2,624 B	2,484 B

2.2 OAuth 2.0, OIDC, and Token Transport Protocols

OAuth 2.0 [**?**] and OpenID Connect 1.0 [**?**] define the industry standard for delegated authorization and identity assertion. In standard OIDC flows, identity

assertions are formatted as JSON Web Tokens (JWTs) [**?**] signed using JSON Web Signature (JWS) [**?**]. JWTs consist of three base64url-encoded segments separated by periods: Header, Payload (Claims), and Signature:

$$\text{JWT} = \text{base64url(Header)} \parallel \text{"."} \parallel \text{base64url(Claims)} \parallel \text{"."} \parallel \text{base64url(Sig)} \quad (1)$$

When passed via HTTP headers, tokens are prefixed with the scheme name:

$$\text{HeaderValue} = \text{"Bearer "} \parallel \text{JWT} \quad (2)$$

Adding a 2,420-byte raw ML-DSA signature adds $\lceil 2420 \times 4/3 \rceil = 3,228$ characters of base64url text, pushing the total token size beyond standard server tolerance limits.

CBOR Web Tokens (CWT) [**?**] provide a binary alternative using Concise Binary Object Representation (CBOR) [**?**]. By replacing string keys with integer mapping identifiers (e.g., claim key `1` for `iss`, `2` for `sub`), CWT substantially reduces formatting overhead compared to JSON.

2.3 Hybrid Cryptography in Identity Systems

Hybrid signatures combine a classical signature scheme with a post-quantum signature scheme [**?**]. The hybrid approach guarantees that an authentication system remains secure as long as *at least one* of the constituent primitives remains unbroken. If a quantum computer breaks Ed25519, ML-DSA-44 maintains signature unforgeability; conversely, if an algorithmic breakthrough compromises ML-DSA-44, Ed25519 protects the current system.

3 DilithiAuth System Architecture

The DilithiAuth system architecture is organized into three primary package modules: `pkg/signer`, `pkg/oauth`, and `pkg/transport`, as illustrated in Fig. **??**.

Figure 1: DilithiAuth High-Level Architecture: Hybrid Key Manager, Token Generation Pipeline, and Sidecar Signature Store (`SigStore`).

3.1 Hybrid Cryptographic Engine (pkg/signer)

The core cryptographic engine is implemented in `pkg/signer/signer.go`. It manages key generation, key rotation, and dual-signing routines.

A `HybridKeyPair` instance encapsulates:

- `Kid`: An 8-byte random byte sequence encoded as an 11-character base64url string.
- `Ed25519Private` / `Ed25519Public`: Standard Ed25519 key pair (64 bytes / 32 bytes).
- `MLDSAPrivate` / `MLDSAPublic`: FIPS 204 ML-DSA-44 key pair (2,560 bytes / 1,312 bytes).

When signing a message payload $M \in \{0,1\}^*$, `Sign(M)` generates both signatures concurrently:

$$\sigma_{\text{Ed25519}} = \text{Sign}_{\text{Ed25519}}(K_{\text{priv}}^{\text{Ed}}, M) \tag{3}$$

$$\sigma_{\text{MLDSA}} = \text{Sign}_{\text{MLDSA44}}(K_{\text{priv}}^{\text{PQC}}, M) \tag{4}$$

The composite result is stored in a `HybridSignature` struct containing both raw byte slices ($\Sigma_{\text{hybrid}} = (\sigma_{\text{Ed25519}}, \sigma_{\text{MLDSA}})$).

Public key publishing is handled via `PublicKeyInfo()`, which structures public keys into JSON Web Key Set (JWKS) format published at `GET /.well-known/jwks.json`. Each JWK entry publishes both public key components under the shared `kid`:

```
{
  "kid": "k9A2fX_1aQ8",
  "kty": "OKP",
  "crv": "Ed25519",
  "x": "...",              // 32-byte Ed25519 (43 chars)
  "mldsa44_pub": "...",    // 1,312-byte ML-DSA (1,750 chars)
  "use": "sig",
  "alg": "EdDSA+MLDSA44"
}
```

3.2 Token Generation Pipeline (pkg/oauth)

The `pkg/oauth` module provides handlers for OIDC discovery (`/.well-known/openid-configuration`) and token exchange. DilithiAuth supports three operational token strategies:

3.2.1 Thin JWT Strategy

Thin JWTs minimize token bloat by including only the 64-byte Ed25519 signature in the JWS signature block. Rather than embedding the 2,420-byte ML-DSA signature directly, the token claims structure (`HybridClaims`) includes a reference hash `pqc_ref`:

$$\text{pqc_ref} = \text{base64url}\left(\text{SHA-256}(\sigma_{\text{MLDSA}})\right) \tag{5}$$

The full σ_{MLDSA} signature is offloaded out-of-band to `SigStore`, keyed by `pqc_ref`.

3.2.2 Jumbo JWT Strategy

Jumbo JWTs embed the raw ML-DSA signature directly inside the token claims as a base64url string claim `pqc_sig`:

$$\text{pqc_sig} = \text{base64url}(\sigma_{\text{MLDSA}}) \tag{6}$$

Jumbo JWTs are stateless and self-contained, allowing offline verification without requiring access to a sidecar store.

3.2.3 CBOR Web Token (CWT) Strategy

CWTs replace JSON string serialization with binary CBOR structure. The claim map uses integer keys defined in RFC 8392:

```
CWT Claims = {
  1: "https://auth.example.com", // iss
  2: "alice",                    // sub
  4: 1770000000,                 // exp
  6: 1769996400,                 // iat
  7: "jti_val_12345",            // cti
  8: "pqc_ref_hash_val"          // pqc_ref
}
```

The final binary CWT envelope is encoded as a two-element CBOR array:

$$\text{CWT}_{\text{raw}} = [\text{CBOR}(\text{Claims}), \sigma_{\text{Ed25519}}] \tag{7}$$

The post-quantum signature σ_{MLDSA} is registered in `SigStore` under the token identifier key `cti` (claim 7).

3.3 Sidecar Signature Store (`pkg/transport`)

To support out-of-band signature resolution for Thin JWTs and CWTs, `pkg/transport` defines the `SigStoreBackend` interface:

```
type SigStoreBackend interface {
    Put(key string, sig []byte)
    Get(key string) ([]byte, bool)
    Delete(key string)
    Len() int
}
```

DilithiAuth provides four pluggable implementations:

1. `MemorySigStore`: In-memory concurrent map with RWMutex locking and defensive slice copies.

2. `RedisSigStore`: Distributed store using Redis key-value storage with native Time-To-Live (TTL) expiration matching token duration.

3. `FileSigStore`: Disk-backed JSON store with atomic file renaming.

4. `PostgresSigStore`: Relational database storage in a `sig_store` table with background TTL cleanup.

To accommodate Jumbo JWTs when enabled, `pkg/transport/middleware.go` provides `NewPQCServer()`, configuring the HTTP server's `MaxHeaderBytes` to 16 KiB (16,384 bytes), overcoming the standard 4 KiB limit.

4 Token Strategy Trade-off Analysis

4.1 Mathematical Size Models

Let S_{claims} denote the raw unencoded byte size of standard identity claims (`iss`, `sub`, `exp`, `iat`, `jti`). The token overhead equations for each strategy are formulated as follows:

For **Jumbo JWT**:

$$\begin{aligned} S_{\text{JumboJWT}} &= S_{\text{Header}} + S_{\text{Payload}} + S_{\text{Sig}} \\ &= \left\lceil \frac{4}{3} S_{\text{hdr_json}} \right\rceil + \left\lceil \frac{4}{3} \left(S_{\text{claims}} + \left\lceil \frac{4}{3} S_{\text{MLDSASig}} \right\rceil \right) \right\rceil \\ &\quad + \left\lceil \frac{4}{3} S_{\text{Ed25519Sig}} \right\rceil \end{aligned} \tag{8}$$

Given $S_{\text{MLDSASig}} = 2,420$ bytes, the embedded base64url string requires 3,227 bytes, yielding a total token payload of $\approx 4,665$ bytes.

For **Thin JWT**:

$$\begin{aligned} S_{\text{ThinJWT}} &= \left\lceil \frac{4}{3} S_{\text{hdr_json}} \right\rceil + \left\lceil \frac{4}{3} \left(S_{\text{claims}} + S_{\text{PQCRef}} \right) \right\rceil \\ &\quad + \left\lceil \frac{4}{3} S_{\text{Ed25519Sig}} \right\rceil \end{aligned} \tag{9}$$

Since $S_{\text{PQCRef}} = 43$ characters (Base64url SHA-256 hash), S_{ThinJWT} remains compact at 419 bytes.

For **CWT (Binary)**:

$$S_{\text{CWT_binary}} = S_{\text{CBOR_hdr}} + S_{\text{CBOR_claims}} + S_{\text{Ed25519Sig_raw}} \tag{10}$$

Because integer key mappings omit claim string names and signatures are stored as raw 64-byte binary slices without base64url text encoding, $S_{\text{CWT_binary}}$ drops to 211 bytes.

The HTTP Header overhead S_{HTTP} for bearer authentication is given by:

$$S_{\text{HTTP}} = \text{len}(\text{"}\textit{Authorization}: \textit{Bearer}\text{"}) + S_{\text{Token}} = 22 + S_{\text{Token}} \tag{11}$$

4.2 Empirical Benchmark Setup

Benchmarks were conducted on a 12th Gen Intel Core i5-1235U processor running Linux `amd64` under Go 1.24. All memory allocations and latencies were measured using Go's standard `testing.B` framework with `-benchmem`.

4.3 Empirical Micro-Benchmark Results

TABLE **??** details the execution latency, memory footprint, and heap allocations for primitive cryptographic operations.

Table 2: Empirical Cryptographic Primitives Micro-Benchmarks

Primitive / Operation	**Latency (μs)**	**Memory (B)**	**Allocs**
Sign Ed25519	28.72 μs	0 B	0
Sign ML-DSA-44	121.58 μs	450 B	3
Sign Hybrid (Ed25519 + ML-DSA)	156.57 μs	3,250 B	6
Verify Ed25519	36.73 μs	0 B	0
Verify ML-DSA-44	27.94 μs	450 B	3
Verify Hybrid (Both)	176.54 μs	450 B	3

As shown in TABLE **??**, ML-DSA-44 verification (27.94 μs) is faster than Ed25519 verification (36.73 μs) due to the efficiency of Number Theoretic Transform (NTT) polynomial multiplications in algebraic lattices. However, hybrid signing requires 156.57 μs due to concurrent signature generation and composite struct packaging.

TABLE **??** presents the comprehensive empirical comparison across all three token strategies.

Table 3: Comprehensive Token Strategy Trade-Off Matrix

Token Strategy	**Token Size**	**HTTP Header**	**Issuance (μs)**	**Verification (μs)**	**Verif.**
Thin JWT	419 B	441 B	276.05 μs	65.31 μs	4
Thin JWT (FastPath)	419 B	441 B	—	43.37 μs	3
Jumbo JWT	4,665 B	4,687 B	194.89 μs	108.33 μs	22
CWT (Binary)	211 B	304 B	257.35 μs	58.20 μs	1
CWT (Base64url)	282 B	304 B	257.35 μs	58.20 μs	1

4.4 Comparative Analysis

1. **Size Ratio & Header Impact**: Jumbo JWT (4,665 bytes) is 11.13× larger than Thin JWT (419 bytes). CWT (211 bytes binary) achieves a 0.50× size ratio compared to Thin JWT (50.8% reduction). While Jumbo JWT headers (4,687 bytes) exceed standard 4 KiB limits, Thin JWT (441

bytes) and CWT (304 bytes) fit comfortably within conventional HTTP header buffers.

2. **Verification Latency & Memory Efficiency**: CWT exhibits the lowest verification latency (58.20 μs) and smallest memory allocation footprint (1,729 B/op across 23 allocations). Thin JWT fast-path verification (validating only Ed25519 at the edge) achieves 43.37 μs latency. Full Jumbo JWT verification requires 108.33 μs and 22,380 B/op due to parsing and decoding the large embedded base64url signature string.

3. **Sidecar Memory Overhead**: For Thin JWTs, `SigStore` maintains a 2,463-byte entry per active token (2,420 bytes ML-DSA signature + 43 bytes `pqc_ref` key). CWT sidecar entries consume 2,442 bytes (2,420 bytes signature + 22 bytes base64url `cti` key).

5 Security Evaluation

5.1 Post-Quantum Lattice Security

DilithiAuth's hybrid scheme combines Ed25519 (providing 128-bit classical security against pre-quantum attacks) with ML-DSA-44 (providing NIST Category 1 post-quantum security). According to NIST FIPS 204 [**?**], ML-DSA-44 provides at least 2^{128} quantum computational operations against Grover's algorithm and is immune to Shor's period-finding algorithm.

Even if an adversary captures historical tokens and subsequently acquires a CRQC capable of breaking Ed25519, the adversary cannot forge an ML-DSA-44 signature. In DilithiAuth, a token is considered fully valid only if both signatures verify successfully, neutralizing HNDL threats.

5.2 Sidecar Integrity Verification via `pqc_ref`

In Thin JWT and CWT modes, post-quantum signatures are retrieved out-of-band from `SigStore`. This architecture introduces the potential threat of sidecar substitution or tampering.

DilithiAuth defends against sidecar tampering through strict cryptographic binding. The `pqc_ref` claim stored inside the signed Thin JWT payload is defined as:

$$\text{pqc_ref} = \text{base64url}(\text{SHA-256}(\sigma_{\text{MLDSA}})) \tag{12}$$

During verification (`pkg/verifier`), after fetching candidate signature bytes $\sigma_{\text{candidate}}$ from `SigStore`, the verifier computes:

$$h_{\text{fetched}} = \text{base64url}(\text{SHA-256}(\sigma_{\text{candidate}})) \tag{13}$$

If $h_{\text{fetched}} \neq \text{pqc_ref}$, the verifier immediately rejects the token prior to executing lattice verification. Because the JWT payload containing `pqc_ref` is signed by Ed25519, an attacker cannot modify `pqc_ref` without invalidating the classical signature.

5.3 Replay Protection and Lifecycle Controls

DilithiAuth enforces comprehensive anti-replay guarantees across all token strategies:

- **Cryptographic Nonces**: Every issued token contains a 16-byte random unique identifier (`jti` / `cti`).
- **Sidecar TTL Alignment**: Entries in `SigStore` inherit exact token TTLs (default 1 hour). When a token expires, its associated ML-DSA signature is automatically purged from Redis/Memory stores.
- **Revocation Checkers**: The verifier interface supports explicit token revocation checks via `RevocationChecker`.

6 Conclusion & Future Directions

This paper presented **DilithiAuth**, a post-quantum Identity Provider mitigating HNDL threats while resolving token bloat. By combining classical Ed25519 with post-quantum ML-DSA-44 in a hybrid architecture and deploying a tri-strategy token pipeline (Thin JWT, Jumbo JWT, CWT) backed by a sidecar signature store (`SigStore`), DilithiAuth achieves quantum-safe identity federation compatible with real-world network constraints.

Empirical evaluation demonstrates that Thin JWTs reduce HTTP header overhead to 441 bytes (compared to 4,687 bytes for Jumbo JWTs) with fast-path verification in 43.37 μs. Binary CWTs achieve maximum compaction at 211 bytes with 58.20 μs verification latency.

Future work will focus on integrating stateful hash-based signatures (NIST SP 800-208 [**?**]) for identity root authorities, hardware security module (HSM) offloading for ML-DSA lattice sampling, and standardization of PQC claims within OpenID Foundation specifications.